

Lista 04 - Introdução a Criptografia

Antônio Erick Freitas Ferreira - 322980

Instituto de Computação - Unicamp - São Paulo - Brasil

18 de junho de 2026

Questão 01.

O esquema não é CPA-Seguro. Note que $\mathbb{Z}_p^*$ é um conjunto como $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$ e, como $p = 2q+1$, temos que metade de $\mathbb{Z}_p^*$ são quadrados e a outra metade não. Isto é, para todo elemento de $\mathbb{Z}_p^*$ temos $\frac{1}{2}$ de chance de um elemento ser um quadrado ou não. Desta forma, um atacante pode fazer o seguinte:

1. O atacante escolhe $m_0 = 0$ e m_1 tal que m_1 não seja um quadrado e envia para o oráculo de cifragem. Note que tanto m_0 e m_1 são elementos de $\mathbb{Z}_p$.
2. O oráculo de cifragem amostra um bit $b \in \{0, 1\}$ e um elemento $r \in \mathbb{Z}_q$ e retorna o par $(c_1, c_2) = (g^r, h^r + m_b \mod p)$.
3. O atacante verifica se c_2 é um quadrado ou não. Se c_2 for um quadrado, o atacante retorna 0, caso contrário, retorna 1.

Note que, se $b = 0$, então $c_2 = h^r + m_0 \mod p = h^r \mod p$ é um quadrado, pois h^r é um quadrado, isto é, $h^r \in G$. Por outro lado, se $b = 1$, então $c_2 = h^r + m_1 \mod p$ nem sempre será um quadrado, pois h^r é um quadrado e m_1 não é um quadrado. Como já discutido, metade dos elementos de $\mathbb{Z}_p^*$ são quadrados e a outra metade não, então c_2 tem $\frac{1}{2}$ de chance de ser um quadrado ou não. Assim, podemos calcular a vantagem do atacante:

$$Adv = |Pr(\text{Atacante retornar } 0 \mid b = 0) - Pr(\text{Atacante retornar } 0 \mid b = 1)|$$

$$Adv = |1 - \frac{1}{2}| = \frac{1}{2} \notin \text{negl}$$

Como Adv não é uma função negligível, o esquema não é CPA-Seguro.

Questão 02.

(Prova por contrapositiva) Se H não é resistente a colisão, então h não é resistente a colisão.

Assumindo que H não é resistente a colisão, então existe $x \neq y$ tal que $H(x) = H(y)$. Entretanto, temos que:

$$H(x) = h(h(x_1)||h(x_2)) \text{ e } H(y) = h(h(y_1)||h(y_2))$$

Seja $w = h(x_1)||h(x_2)$ e $z = h(y_1)||h(y_2)$. Se $w \neq z$, então $h(w) = h(z)$ e isso é uma colisão. Se $w = z$, então $h(x_1) = h(y_1)$ e $h(x_2) = h(y_2)$. Entretanto, $x \neq y$, o que garante que $x_1 \neq y_1$ ou $x_2 \neq y_2$, implicando que em alguma dessas igualdades há uma colisão. Como em ambos os casos há uma colisão em h , então h não é resistente a colisão.

Portanto, podemos concluir que se h for resistente a colisão, então H será resistente a colisão.

Questão 3.

Item A)

Ao aplicarmos o CRT(s_p, s_q) temos que:

$$\begin{cases} s \equiv s_p \pmod{p} \\ s \equiv s_q \pmod{q} \end{cases}$$

Demonstração para o primeiro caso (mod p):

$$s \equiv s_p \pmod{p} \text{ (elevando ambos os lados a } e \text{)}$$

$$s^e \equiv s_p^e \pmod{p} \quad (s_p = m^{d_p} \pmod{p})$$

$$s^e \equiv m^{d_p e} \pmod{p} \quad (d_p \equiv d \pmod{p-1} \rightarrow d_p e \equiv de \pmod{p-1})$$

$s^e \equiv m^{de} \pmod{p-1} \pmod{p}$ (como d é o inverso de e em modulo $\phi(n)$, temos que $de \equiv 1 \pmod{p-1}$, o que implica que $de = k(p-1) + 1$)

$$s^e \equiv m^{k(p-1)+1} \pmod{p}$$

$$s^e \equiv m \cdot (m^{p-1})^k \pmod{p} \quad (\text{aplicando o pequeno teorema de Fermat temos que } m^{p-1} = 1)$$

$$s^e \equiv m \cdot (1)^k \pmod{p}$$

$$s^e \equiv m \pmod{p}$$

De forma inteiramente análoga, demonstra-se para o segundo caso que $s^e \equiv m \pmod{q}$. Como p e q são números primos distintos (e, portanto, coprimos entre si), pelo Teorema Chinês dos Restos (CRT) conclui-se que:

$$s^e \equiv m \pmod{pq} \implies s^e = m \in \mathbb{Z}_n$$

o que prova que s é uma assinatura RSA válida.

Item B)

Como (s, m) são publicados como o usual e sabemos que houve um erro no calculo com s_p , temos que:

$$\begin{cases} s^e \not\equiv m \pmod{p} \\ s^e \equiv m \pmod{q} \end{cases}$$

Como demonstrado no item A, sabemos que $s^e \equiv m \pmod{q}$ (e $s^e \not\equiv m \pmod{p}$, como dito no enunciado), o que implica que $s^e = k \cdot q + m$ e que $s^e - m = k \cdot q$. Note que $s^e - m$ é um multiplo perfeito de q , além disso, como (s, m) e a chave pública (n, e) são conhecidas, qualquer pessoa seria capaz de calcular tais valores. Portanto, alguém pode fazer os seguintes passos para recuperar p e q .

1. Calcule $w = s^e - m$ e obtenha $q = \gcd(w, n)$, onde $\gcd$ é o algoritmo de Euclides. Como w é um multiplo de q e $n = p \cdot q$, o maior divisor comum entre w e n será q
2. Calcule $p = \frac{n}{q}$

Questão 4.

Eve tem acesso aos 3 criptogramas e sabe que

$$\begin{cases} c_1 \equiv m^e \pmod{n_1} \\ c_2 \equiv m^e \pmod{n_2} \\ c_3 \equiv m^e \pmod{n_3} \end{cases}$$

Ao aplicar $x = CRT(c_1, c_2, c_3)$, onde $x \equiv m^3 \pmod{n_1 \cdot n_2 \cdot n_3}$. Como Eve também sabe que $m^3 < n_1 \cdot n_2 \cdot n_3$, então também é sabido que não houve uma volta no modulo, isto é, $m^3 \pmod{n_1 \cdot n_2 \cdot n_3} = m^3$. Com isso, basta que Eve calcule $m = \sqrt[3]{x}$.

Questão 5.

Item A)

Usar DFactor para resolver DPrimes. Para esta resolução, estou desconsiderando números primos triviais, ou seja, para qualquer $x > 3$ o algoritmo deve funcionar.

Algorithm 1: DPrimes

Input: $x \in \mathbb{N}$

Output: 1 se x é um número primo e 0 caso contrário.

```

1  $y \leftarrow \lfloor \sqrt{x} \rfloor$ 
2 if DFactor( $y, x$ ) = 1 then
3   return 0
4 return 1
```

Item B)

Algorithm 2: IntegerFactor

Input: $x \in \mathbb{N}$
Output: Uma lista Q contendo os fatores de x .

```
1  $Q \leftarrow []$ 
2 while  $x > 1$  do
3    $low \leftarrow 2$ 
4    $high \leftarrow x$ 
5    $factor \leftarrow x$ 
6   while  $low \leq high$  do
7      $mid \leftarrow \frac{low+high}{2}$ 
8     if  $D\text{Factor}(mid, x) = 1$  then
9        $factor \leftarrow mid$ 
10       $high \leftarrow mid - 1$ 
11     else
12        $low = mid + 1$ 
13   Insira  $factor$  em  $Q$ 
14    $x \leftarrow \frac{x}{factor}$ 
15 return  $Q$ 
```

Item C)

Isso implicaria que seria capaz de fatorar números apenas conseguindo distinguir se um número é primo ou não (o que já acontece hoje com alguns algoritmos famosos, como o Miller-Rabin). Não, essa redução não existe (pelo menos não comprovadamente), além disso, se existisse, toda a segurança do RSA seria jogada fora, por isso é muito improvável que tal redução exista.

Questão 6.

Como a questão pede para que seja proposto um Dec, podemos assumir que temos acesso a chave secreta $sk = (p, q)$. Para o Dec, faça o que segue.

1. Receba (c_1, c_2) e calcule as raízes quadradas modulo p e q , as combine e obtenha as 4 raízes quadradas $R = \{r_1, r_2, r_3, r_4\}$ e $R' = \{r'_1, r'_2, r'_3, r'_4\}$ para c_1 e c_2 respectivamente.
2. Para todo $r'_i \in R'$ e $r_j \in R$, se $r'_i - r_j \equiv 1 \pmod{p \cdot q}$ retorne r_j .

Discussão sobre corretude: Sabemos que os elementos em R são combinações das raízes quadradas de c_1 , podendo ser escritas como $r_1 = (m_p, m_q)$, $r_2 = (-m_p, m_q)$, $r_3 = (m_p, -m_q)$, $r_4 = (-m_p, -m_q)$. Além disso, os elementos de R' podem ser escritos como $r'_1 = (m_p + 1, m_q + 1)$, $r'_2 = (-(m_p + 1), m_q + 1)$, $r'_3 = (m_p + 1, -(m_q + 1))$, $r'_4 = (-(m_p + 1), -(m_q + 1))$.

Note que r_1 é a mensagem correta. Além disso, $r'_1 - r_1 = (m_p + 1 - m_p, m_q + 1 - m_q) = (1, 1)$. Qualquer outro par subtraído terá valor diferente de $(1, 1)$, por isso, o resultado deixa de ser ambíguo e passa a ser determinístico e correto.

Questão 07.**Item A)**

Como Bob tem a chave pública de Oscar acreditando ser a do CA, Oscar pode fazer o seguinte para evitar que Bob descubra que possui a chave pública errada:

1. Oscar intercepta ambas as mensagens (1) e (2)
2. Oscar computa $x = ((1), (2)) = (ID_B, pk_B)$
3. Oscar computa $\sigma = \text{Sign}(x, sk_O)$, onde sk_O é a chave privada de Oscar (também chamada de chave de assinatura) e retornar a Bob $cert_B = (\sigma, x)$.

Como Bob possui a chave pública de Oscar pk_O , Bob verifica $cert_B$ utilizando a função $\text{Verify}(cert_B[0], cert_B[1], pk_O)$, como isso é um certificado válido, então Verify retornará 1 e Bob acreditará que está tudo bem.

Item B)

Para que Oscar possa estabelecer uma sessão chaveada com Bob usando DFKE de forma que Bob pense que está conversando com Alice, Oscar pode seguir os passos a seguir.

1. Oscar intercepta o Client Hello (que contém os algoritmos de cifração suportados, junto da chave pública g^b) de Bob.
2. Oscar gera (o, g^o) , computa o segredo DH $= (g^b)^o$ e roda um protocolo para derivar a chave DH em várias chaves para os outros algoritmos.
3. Oscar agora envia um pacote para Bob contendo o Server Hello (que contém os algoritmos de cifração selecionados e a chave pública g^o), um certificado digital falso contendo as informações de Alice mas assinado com a chave de Oscar, assinaturas digitais sobre o Client Hello, Server Hello e o certificado, e os MAC sobre o Client Hello, Server Hello, certificado e assinaturas digitais.
4. Bob irá verificar o certificado digital que recebeu, como ele tem a chave de Oscar no lugar, essa verificação será aprovada. Em seguida, Bob remove a vk do Oscar de dentro do certificado. Agora Bob verifica as assinaturas, computa a chave secreta DH $= (g^o)^b$, roda o mesmo protocolo para derivar a chave DH em várias chaves para os outros algoritmos e, por fim, verifica os MAC usando as chaves derivadas.

Como todas as etapas de verificação entre Bob e Oscar serão aprovadas, Oscar agora tem uma sessão chaveada com Bob e Bob pensa que esta sessão é com Alice.